Explainable AI and Model Compression: from the notebook to an MLOps workflow (MANOLO Innovation Lab)

A) OBJECTIVES

1. Understand what Explainable AI (XAI) and Model Compression are, and why they matter for trustworthy and efficient AI.
2. Apply an explainability method (Grad-CAM) to a CNN trained from scratch, and use it to inspect correct and wrong predictions.
3. Apply post-training compression (int8 quantization and magnitude pruning) and measure its cost in accuracy, size and latency.
4. Understand the role of MLOps tools (MLflow, MANOLO) and run the same evaluation, explanation and compression steps as a reproducible MANOLO workflow.

Optional task goals: a second or third dataset, other CAM methods, compress-then-explain, structured pruning.

B) INTRODUCTION: This session is an innovation lab built with the tools developed in the European project MANOLO (Trustworthy Efficient AI for Cloud-Edge Computing, Horizon Europe GA 101135782), in which Bitbrain is a partner. A test accuracy tells us how often a model is right, not why, and a model that works on a GPU may not fit on the device where it must run. You will first train your own image classifier (a ResNet-18 from scratch), explain it with Grad-CAM and compress it with quantization and pruning, all in a notebook. Then you will hand the same model and data to MANOLO, which runs these steps as a tracked workflow on remote compute nodes and logs every metric (including energy and CO_2) in MLflow.

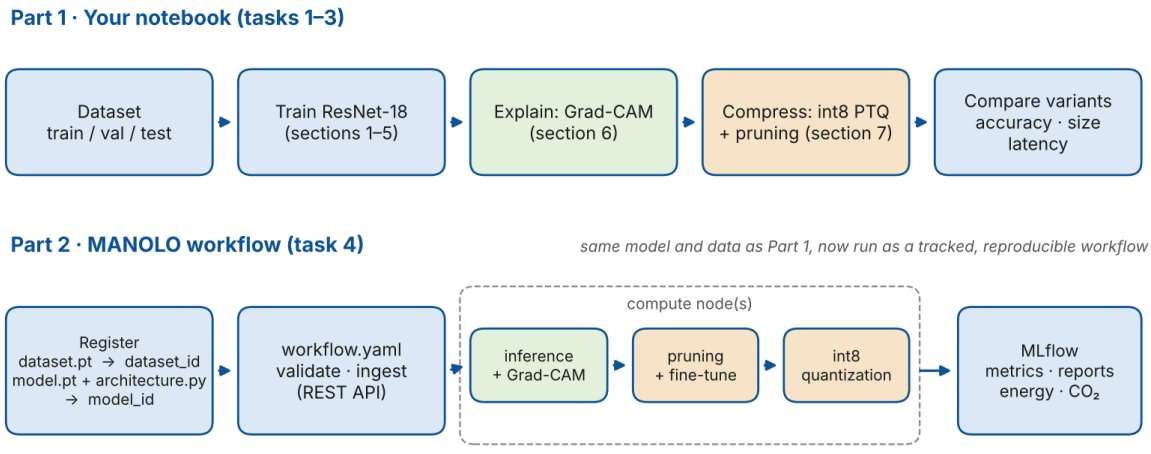


Figure 1. Structure of the lab: the notebook (tasks 1–3) and the MANOLO workflow (task 4).

The session follows four steps:

- **Introduction**: short briefing on XAI and Model Compression (slides **mc_xai_intro_manolo.pptx**).
- **Exploration**: train, explain and compress your own model in the notebook, at your own pace (sections 1–7).
- **Hands-on with MANOLO**: during the lab session. Register your dataset and model on the platform, launch a workflow and compare its results with your own (section 8).

PREVIOUS TO THE SESSION: Choose your dataset and prepare your environment (Google Colab or your own computer), so that the session can be used for the experiments.

1. Choose a dataset. The materials include three binary image classification datasets in already split into train/val/test. Pick and download at least one of them from Moodle:

- **Wildfire**: satellite images with and without wildfire (`wildfire-prediction-dataset`, ~43,000 images: the largest and slowest to train).
- **Rice blast**: rice leaves, healthy or with blast disease (`riceguard-19k-cleaned-dataset`, ~9,800 images).
- **Chest X-ray**: normal or pneumonia (`augcxr-dataset`, ~10,000 images).

You must analyse **ONE dataset of your choice** in the mandatory tasks. Analysing a second or third dataset is optional (see section E). Each dataset follows this format (the class names are the folder names):

```
DATASET_DIR/
├── train/
│   ├── class_a/      image files (.jpg, .png, ...)
│   └── class_b/
├── val/
│   └── ...
└── test/
    └── ...
```

2. Prepare the environment. The files you need are in the **innovation:lab compressed file**: **workshop_participant.ipynb** (your notebook), **lab_utils.py** (helper functions), **architecture.py** (the model, in the format MANOLO expects), **workflow.yaml** (the workflow you will submit) and **requirements.txt**.

Option A (recommended): Google Colab + Google Drive.

- Copy the **innovation_lab** folder and **the selected datasets** to your Google Drive, e.g. into `MyDrive/lab4/`. Open **workshop_participant.ipynb** from Drive with Google Colab.
- Select a GPU: **Runtime > Change runtime type > T4 GPU**. Training on the CPU also works, but it is much slower.

- Add a first cell that mounts Drive, unzips your dataset to the local disk of the Colab machine, moves into the **innovation_lab** folder (so that the notebook can import `lab_utils.py` and `architecture.py`) and installs Grad-CAM:

```
from google.colab import drive
drive.mount('/content/drive')
!unzip -q -n /content/drive/MyDrive/lab4/YOURDATASET.zip
YOURDATASET/*' -d /content/data
%cd /content/drive/MyDrive/lab4/innovation_lab
!pip install -q grad-cam
```

- Then set `DATASET_DIR = Path("/content/data/YOURDATASET")` (change the name to your dataset) and `NUM_WORKERS = 2`. Reading thousands of small images directly from Drive is very slow: keep the zip on Drive and unzip it to `/content/data` at the start of every Colab session (it takes a few minutes).
- Do not run `pip install -r requirements.txt` on Colab: it already provides PyTorch, torchvision, pandas and matplotlib. Your `outputs/` folder is written inside **innovation_lab** on Drive, so the trained model survives a disconnection (set `TRAIN = False` to reload it).

Option B: your own computer.

- Python 3.10 or later. From the **innovation_lab** folder run `pip install -r requirements.txt`. On a CPU-only machine, install `torch` and `torchvision` first from download.pytorch.org/whl/cpu to avoid the large CUDA download. Unzip the dataset you chose and start Jupyter from the **innovation_lab** folder.

C) EVALUATION: Submit your notebook to Moodle **by the end of the session.** Upload the **workshop_participant.ipynb** with all your code and answers (in Colab: **File > Download > Download .ipynb**), together with the **workflow.yaml** you submitted to MANOLO (`outputs/<RUN_NAME>/workflow.yaml`). MAKE SURE the **notebook saves the outputs of your cells** (tables, heatmaps and plots must be visible without re-running it). If you analyse more than one dataset, submit one notebook per dataset (e.g. `lab4_rice.ipynb`, `lab4_xray.ipynb`).

The following will be evaluated:

- Correctness and extent of the work. (NO NEED for HIGH accuracies! We want correct experiments and a sound analysis.)
- Correctness in the use of technical terms in the answers.
- Analysis and discussion of the results (explanations and compression trade-offs).
- Organization and cleanliness of the code.

REMEMBER:

- **To opt for the maximum grade, REQUIRED to submit all mandatory tasks + the optional task.**

- Work in groups of two as in the previous labs. Use your group name and the dataset in RUN_NAME (e.g. team-a-wildfire): it names your output folder and your assets on MANOLO.

D) DESCRIPTION OF MANDATORY TASKS for LAB4:

Follow the notebook **workshop_participant.ipynb** section by section: **complete the cells marked TODO** (each one comes with a hint) and **answer the questions below in a markdown cell at the end of the corresponding section**.

1. Baseline CNN. Notebook sections **1 to 5**.

- 1.1 Which dataset did you choose? Did you use GRAYSCALE = True or False, and why?
- 1.2 Are the classes balanced? Why is accuracy alone not enough to judge the model?
- 1.3 Report the test accuracy, the per-class recall and the confusion matrix. Do the learning curves show over- or under-fitting?

2. Explainability (XAI). Notebook section **6** (TODO 1–5).

- 2.1 Does the model look at the object of interest, or at the background, the borders or text?
- 2.2 How do the heatmaps change from layer2 to layer4? What do you gain and lose with depth?
- 2.3 Why are the misclassified images wrong: ambiguous images, or a model looking at the wrong place?
- 2.4 Try another method of pytorch-grad-cam (e.g. GradCAMPlusPlus, HiResCAM, ScoreCAM or EigenCAM). Do the explanations agree with Grad-CAM?

3. Compression. Notebook section **7** (TODO 6–13).

- 3.1 Which compression gives the best trade-off for your dataset? Where does the accuracy collapse?
- 3.2 Why does pruning change neither the size nor the latency, while quantization changes both?
- 3.3 Does fine-tuning recover the accuracy? Up to which sparsity is it enough?
- 3.4 How many calibration batches does the int8 model need?

4. MANOLO. Notebook section **8**, completed together during the MANOLO presentation (controller URL and credentials given by the instructors).

- 4.1 Compare MANOLO's results (accuracy, compressed models, Grad-CAM report) with yours from tasks 2 and 3. Are they consistent?
- 4.2 What does the workflow measure that your notebook does not (e.g. energy and CO_2)?

- 4.3 What does an MLOps framework such as MANOLO add to the notebook approach for a real deployment?

E) Optional task:

(1.5 points) Choose **one** of the following and analyze the results in the notebook:

- **A second (or third) dataset.** Repeat tasks 1 to 3 on another dataset, in a copy of the notebook and with a different RUN_NAME. Compare the datasets: does colour (GRAYSCALE) matter, where does the model look, and is one dataset more robust to compression than the other?
- **Compress, then explain again.** Recompute Grad-CAM on the pruned models (the int8 model has no gradients) for the same images. Do the explanations stay stable as sparsity grows? Do the errors concentrate on particular classes?
- **Structured pruning.** Remove whole filters with Torch-Pruning, or set `pruning_method: structured` in workflow.yaml, and compare size and latency with unstructured pruning.
- **A sanity check for the explainer.** Randomize the weights of the last layers and check whether the Grad-CAM heatmaps change (Adebayo et al., 2018).

F) TO LEARN MORE

Grad-CAM

- Original paper (Selvaraju et al., 2017)
- pytorch-grad-cam library and its tutorials and method comparison
- Interpretable Machine Learning (C. Molnar), online book

Quantization

- PyTorch quantization documentation and static quantization tutorial
- A white paper on neural network quantization (Nagel et al., 2021)

Pruning

- PyTorch pruning tutorial
- Learning both weights and connections (Han et al., 2015)
- What is the state of neural network pruning? (Blalock et al., 2020)